



Payload Decoding Guide

MeteoRain IoT Pro Gen2 | How to parse the payloads
Firmware 1.02.012

1. What this document is for

This is a decoder-writer's guide for the MeteoRain IoT Pro Gen2. It covers every LoRaWAN payload the rain gauge sends, in the order you need them to write a parser: how the bits are laid out, how each field converts to a physical value, and the encodings that produce wrong readings if you decode them naively. Section 9 adds the alarm-settings downlink, which is bit-packed the same way, because building it is the same skill in reverse, and section 10 covers the short command replies. A complete, tested JavaScript decoder is included in section 11.

The document is self-contained: everything needed to decode the gauge's uplinks and to build an alarm-settings command is here. The field layouts were taken from the firmware itself and the decoder in section 11 was verified against it, so where this guide disagrees with an older payload sheet, this guide describes what firmware 1.02.012 actually does.

1.1 What this document covers

Port	Message	Size	Direction	Section
1	Periodic measurement	6 B	uplink	4
3	Alarm event	4 B	uplink	5
2	Service message MR1S - device identity	12 B	uplink	6
2	Service message MR2S - alarm settings readback	12 B	uplink	7
2	Service message MR0S - NOT IMPLEMENTED, see 8	12 B	uplink	8
4	Retransmission of a stored payload	6 or 4 B	uplink	4 / 5
50	Reply to a configuration command	2 - 6 B	uplink	10
10	Alarm-settings command	4 B	downlink	9

A payload on port 4 is a byte-for-byte replay of one the gauge sent earlier on port 1 or port 3, resent on request. It carries no extra header, so decode it by length: 6 bytes uses the periodic layout of section 4, 4 bytes the alarm layout of section 5. The index field inside identifies which original message it is.

The remaining downlink ports - radio configuration, reset, time sync, measurement period, retransmission requests - do not carry bit-packed payloads and are described in the Downlink Command Guide rather than here. The one exception is the alarm-settings command on port 10, which is packed exactly like an uplink and so belongs in this document.

2. The parsing model

Every payload is one continuous bitstream. Fields are packed most-significant-bit first, back to back, with no padding or alignment between them, so most fields straddle byte boundaries. In the periodic payload only the index field happens to start and end on a byte.

Do not try to parse these payloads by slicing bytes. A byte-oriented parser works for the index field and then silently produces nonsense for everything after it. Read bit ranges.

2.1 Four steps

Step	What to do
1	Take the raw bytes and treat them as one bitstream, bit 1 being the most significant bit of byte 0
2	For each field in table order, read its width in bits, most significant bit first, into an unsigned integer - call it raw
3	The physical value is $\text{raw} * \text{resolution} + \text{minimum}$, using the resolution and minimum from the field table
4	Then apply the conversions of section 3 - the rain time field is not seconds, the click counter is not rainfall, and the battery bit is not a voltage

2.2 A payload decoded by hand

Periodic payload 0380DAD80E70 on port 1. Writing the 6 bytes out as 48 bits and cutting them at the field widths from section 4 gives:

Bits	Field	Bits read	raw	Value	Unit
1-8	Index	00000011	3	3	
9	Battery in slot	1	1	1	
10-21	Rain clicks	000000011011	27	27	clicks
22-31	Rain min time	0101101100	364	364	728/sqrt(s)
32-43	Rain intensity corr	000001110011	115	1.15	clicks
44	Internal temp > 2 deg C	1	1	1	
45-48	Debug flags	0000	0	0	

Two fields are not finished at this point. The rain min time of 364 is not a number of seconds - see 3.2 - and the 27 clicks are a lifetime total, not the rain that fell during this period - see 3.1. Only the intensity correction is a straight scaling: $115 \times 0.01 = 1.15$ clicks.

3. Four things that break naive decoders

3.1 Rain clicks is a lifetime counter, not rainfall in the period

The rain clicks field is the gauge's running total, not the rain that fell since the last uplink. A decoder that multiplies it by the bucket size and publishes the result reports a number that only ever grows, then collapses to zero - it is not rainfall and it is not cumulative site rainfall either.

The counter is 12 bits and revolves: it counts 0 to 4095 and then returns to 0. Rainfall between two payloads is

Quantity	Formula
Clicks in the interval	$(\text{clicks_now} - \text{clicks_prev}) \bmod 4096$
Rainfall in the interval	$\text{clicks_in_interval} * \text{bucket_mm}$
With the under-catch correction	$(\text{clicks} + (\text{corr_now} - \text{corr_prev}) \bmod 10.23) * \text{bucket_mm}$

bucket_mm is the rain gauge resolution in mm per click. It is a property of the gauge fitted, not of the payload - the device never transmits it. Set it once in the decoder.

A decrease in the counter does not always mean it wrapped. The counter can also be set to an arbitrary value by downlink command 121, and it is restored from non-volatile memory after a reboot. Treat a large unexplained jump as a gap in the series rather than as rainfall.

3.2 Rain time is transmitted as $728 / \sqrt{\text{seconds}}$

The rain time field is not an interval. The firmware transmits $728 / \sqrt{t}$ where t is the shortest interval between two tipping-bucket clicks during the period, which puts the resolution where it matters - heavy rain, short intervals. A transmitted 0 means no two clicks were recorded, not a zero interval.

Quantity	Formula
Interval between clicks	$t_seconds = (728 / \text{raw})^2$
Rain rate, from the interval	$\text{rate_mm_h} = 3600 / t_seconds * \text{bucket_mm}$
Rain rate, direct from raw	$\text{rate_mm_h} = 3600 * \text{bucket_mm} * \text{raw}^2 / 529984$

The direct form avoids the square root and is what the decoder uses; 529984 is 728 squared. Note that this is the PEAK rate during the period, taken from the single shortest gap between clicks - it is not an average over the period.

raw	Interval	Rate at 0.1 mm	Rate at 0.2 mm	Rate at 0.25 mm	mm/h per LSB (0.2 mm)
0	not measured	-	-	-	-
104	49.00 s	7.35	14.69	18.37	0.28
230	10.02 s	35.93	71.87	89.83	0.63
364	4.00 s	90.00	180.00	225.00	0.99
728	1.00 s	360.00	720.00	900.00	1.98
1023	0.51 s	710.87	1421.74	1777.18	2.78

Resolution degrades sharply at high rain rates: near the top of the range one LSB is worth about 2.8 mm/h with a 0.2 mm gauge. Do not present decoded heavy rain rates to more precision than that.

3.3 The battery bit is not a voltage

The periodic payload carries one bit of battery information. It is 1 only when the battery voltage falls inside the single 200 mV slot selected by index % 5:

index % 5	Battery slot the bit reports on
0	3300 - 3500 mV
1	3500 - 3700 mV

index % 5	Battery slot the bit reports on
2	3700 - 3900 mV
3	3900 - 4100 mV
4	4100 - 4300 mV

So a 1 means "the voltage is inside this payload's slot" and a 0 means "it is not" - which says nothing about which other slot it is in. To recover a voltage a server must watch five consecutive indices and note which one reported 1. Until then the battery state is unknown, and a device that has just joined cannot report a voltage at all for the first few uplinks.

If every one of five consecutive indices reports 0, the voltage is outside 3300-4300 mV entirely - treat that as a fault rather than as missing data.

3.4 All-ones means the value did not fit

When a value is NaN or falls outside the range a field can encode, the firmware does not clamp it and does not omit the field: it writes every bit of that field to 1. On this device only two fields can reach that state, and they mean different things.

Field	Bits	All-ones raw	What it means
Rain min time / interval	10	1023	Saturation, not a fault. Clicks came faster than 0.51 s apart. The true rate is at or above 1421 mm/h with a 0.2 mm gauge - report it as a lower bound, not as an exact reading
Rain intensity correction	12	4095	Out of range. The firmware keeps this accumulator below 10.23, so 4095 (40.95) should never be transmitted - treat it as invalid and emit null

The rule does not apply to counters, flags and identifiers - the index, rain clicks, the temperature and debug bits, and the device address - where an all-ones pattern is a legitimate value. A rain clicks field of 4095 is a full counter about to wrap, not an error.

4. Port 1 - periodic measurement, 6 bytes

The routine uplink, sent every 2 to 10 minutes. Also appears on port 4 as a retransmission. Uses all 48 bits.

Bits	Field	W	Min	Res	Decoded unit	Notes
1-8	Index	8	0	1	-	Revolving 0-255. Identifies the payload for retransmission requests
9	Battery in slot	1	0	1	-	See 3.3 - not a voltage
10-21	Rain clicks	12	0	1	clicks	Lifetime revolving counter, NOT per period - see 3.1
22-31	Rain min time	10	0	1	-	Shortest gap between clicks in the period. Apply 3.2. 0 means not measured
32-43	Rain intensity corr	12	0	0.01	clicks	Revolving accumulator of missed clicks, 0.00-10.23 - see 4.1
44	Internal temp > 2 deg C	1	0	1	-	Frost warning flag - see 4.2. Not a temperature
45-48	Debug flags	4	0	1	-	See 4.3

4.1 The intensity correction is a second revolving counter

A tipping-bucket gauge under-catches in heavy rain, because water keeps arriving while the bucket is tipping. The firmware estimates the clicks lost this way and accumulates them, as a fractional number of clicks, in this field. It is a revolving accumulator exactly like the click counter, but it wraps at 10.23 rather than at 4095, and it is only advanced while the rain is heavy enough to need correcting.

Use it the same way: difference two consecutive values, wrap negatives by adding 10.23, and add the result to the click difference before multiplying by the bucket size. A decoder that reports the raw field as "correction" without differencing it reports a meaningless running total.

The correction curve itself is a straight line between two rain intensities, both configurable by downlink command 124 and readable back on port 50 - see section 10. The factory defaults add no correction at 350 clicks per hour and 6.7% at 3000 clicks per hour, which with a 0.2 mm gauge is 70 mm/h and 600 mm/h respectively. Below the lower breakpoint no correction accumulates at all.

4.2 The temperature bit is a frost warning, not a measurement

Bit 44 is 1 when the gauge's internal temperature is above 2 deg C and 0 otherwise. It exists to warn that readings may be affected by freezing - a frozen funnel or bucket stops delivering clicks, so a 0 here alongside zero rainfall is not evidence that it was dry. The gauge transmits no temperature value of any kind.

The bit reads 0 before the first successful temperature measurement, and if the temperature sensor stops responding the bit holds the last value that was read rather than flagging an error. Do not use it as a sensor-health signal.

4.3 Debug flags

Value	Meaning
0	Normal
1	An alarm payload was sent during this period
2	Non-volatile memory read or write failed - the click counter in this payload may be stale or wrong

The firmware overwrites this field rather than combining flags into it, so only the most recent event is reported and the value is never 3. Within a period it is the last event that survives, with two fixed exceptions: a memory error and an alarm raised by the SAME rain click always report the alarm, and a memory read that fails while the payload is being built overwrites everything, clearing the alarm flag with it. So a 1 rules out a memory failure at build time, but a 2 tells you

nothing about whether an alarm was also sent during the period.



5. Port 3 - alarm, 4 bytes

Sent as soon as the rain rate crosses the configured threshold, without waiting for the next periodic uplink. It carries the instantaneous click interval that triggered it rather than a period figure. Also appears on port 4 as a retransmission. Uses all 32 bits.

Bits	Field	W	Min	Res	Decoded unit	Notes
1-5	Index	5	0	1	-	Revolving 0-31, a separate counter from the periodic index
6-17	Rain clicks	12	0	1	clicks	Lifetime revolving counter, as in 3.1
18-27	Rain interval	10	0	1	-	The interval that triggered the alarm. Apply 3.2
28-32	Debug flags	5	0	1	-	Always 0 in this firmware

5.1 What triggers an alarm

One condition only: the measured interval between two consecutive clicks is SHORTER than the configured threshold interval - that is, the rain is falling faster than the configured rate. There is no low-rain alarm and no other alarm type on this device, which is why the payload carries no alarm-type field.

After an alarm the gauge holds off further alarms for the configured snooze time, counted down in 10-second steps. Rain keeps being counted during the snooze - only the alarm message is suppressed - so no rainfall is lost, and the periodic payload still reports it.

The alarm index revolves 0-31, not 0-63. It is also the index used to request a retransmission of a stored alarm payload, and a request for an index of 32 or higher is rejected.

6. Port 2 - MR1S service message, 12 bytes

Device identity. Sent in response to downlink command 120 with parameter 01, and once automatically after the device joins the network. Uses 72 of the 96 bits; bits 73-96 are zero padding, not a field.

MR0S, MR1S and MR2S are all 12 bytes on port 2. Read the first 2 bits to tell them apart: 0 is MR0S, 1 is MR1S, 2 is MR2S. Do not branch on length.

Bits	Field	W	Notes
1-2	Message type	2	Always 1 for MR1S. This is the discriminator
3-7	Hardware type	5	0 test, 1 MeteoHelix, 2 MeteoWind, 3 MeteoRain, 4 MeteoAG
8-10	FW major	3	Firmware version, first component. 0-7
11-16	FW minor	6	Second component, 0-63. Render zero-padded to 2 digits
17-24	FW patch	8	Third component, 0-255. Render zero-padded to 3 digits
25-32	Debug	8	Always 0 in this firmware
33-64	Device address	32	LoRaWAN DevAddr. Render as 8 hex digits
65-72	Payload period	8	Measurement and uplink period in minutes, 2-10
73-96	(padding)	24	Zero fill, not a field

The three version components are separate fields, so 1, 2, 12 renders as 1.02.012 - zero-pad the minor to two digits and the patch to three to match how the gauge names its own version.

MR1S is the only message that reports the uplink period, which is worth reading before you decide that a device has gone quiet: a gauge configured for 10 minutes and one configured for 2 look very different in a dashboard. Unlike the MeteoHelix, this device carries no sensor serial number - the rain gauge has no digital sensor head to identify.

7. Port 2 - MR2S service message, 12 bytes

The alarm settings currently in effect. Sent only in response to downlink command 120 with parameter 02, and it is the only way to confirm that an alarm-settings downlink was accepted, because that command sends no reply of its own.

MR2S carries just 21 bits of fields inside a 12-byte frame; bits 22-96 are zero padding. Older payload documentation describes an 11-bit debug-flags field at bits 22-32. The firmware never writes it, so it always reads zero. See 12.4.

Bits	Field	W	Min	Res	Decoded unit	Notes
1-2	Message type	2	0	1	-	Always 2 for MR2S
3-11	All-alarms snooze	9	0	120	s	Hold-off after an alarm, 120-61320 s
12-21	Rain click interval	10	0	1	-	Alarm threshold. Apply 3.2 to read it as seconds
22-96	(padding)	75	-	-	-	Zero fill, not a field

The threshold is stored and reported in the same $728/\sqrt{t}$ form as the measurements, so it can be compared against the rain interval field of a periodic or alarm payload without converting either: a raw measurement LARGER than the raw threshold means rain heavier than the threshold, and that is what raises an alarm.



8. Port 2 - MR0S service message - NOT IMPLEMENTED

MR0S is NOT IMPLEMENTED on the MeteoRain IoT Pro Gen2. The device does not carry the sensors this message was designed to report, so nothing in it is a measurement. It is documented here only so that a decoder can recognise the message and discard it.

The message nevertheless exists in the firmware, inherited from the shared service-message family used across the MeteoHelix, MeteoWind and MeteoAG products. If it is requested - downlink command 120 with parameter 00 - the gauge will assemble a well-formed 12-byte frame on port 2 and transmit it. Every field in that frame is a compile-time constant, identical on every unit ever shipped.

The danger is that the frame looks completely valid. A decoder written from the generic service-message layout will publish readings that were never measured, and every device in the fleet will report exactly the same ones. Never send command 120 with parameter 00 in normal operation, and discard the message if one arrives.

The layout below is given only so that the message can be identified and its bits skipped. The 64 bits at positions 3-66 are deliberately NOT documented: they hold fixed constants that were never measured, and printing them here would only invite someone to use them. Do not build any feature on any part of this message.

Bits	Field	W	Min	Res	Fixed value it always reports
1-2	Message type	2	0	1	0 - the discriminator
3-66	(not implemented)	64	-	-	Fixed constants, deliberately not documented. Skip these 64 bits
67-74	Inclination X	8	0	0.25	8.00 deg
75-82	Inclination Y	8	0	0.25	2.50 deg
83-90	Inclination Z	8	0	0.25	0.50 deg
91-96	Debug flags	6	0	1	0

9. Port 10 - alarm-settings downlink, 4 bytes

This is the one downlink built the same way as the uplinks. Its 32 bits are the same 21 fields as MR2S (section 7) followed by an 11-bit debug field, filling 4 bytes exactly. If you can decode MR2S you can already encode this.

Property	Value
Port	10
Payload	4 bytes
Reply	None - see 9.3
Effect	Replaces both alarm settings at once. The gauge keeps them through a power cycle
Device reset	No

The gauge is a Class A LoRaWAN device: it only listens briefly after it transmits. A downlink queued on the network server is delivered after the gauge's next uplink, which can be up to the configured measurement period away - by default 10 minutes, plus up to 29 seconds of transmit spreading. Queue one command at a time and wait for it to be delivered.

Bits	Field	W	Min	Res	Valid range the firmware accepts
1-2	Type	2	0	1	Ignored entirely. Send 0
3-11	All-alarms snooze	9	0	120	raw 1-511, giving 120 - 61320 s
12-21	Rain click interval	10	0	1	raw 1-1023, giving 529984 - 0.51 s. 0 selects the 4.0 s default
22-32	Debug flags	11	0	1	Ignored entirely. Send 0

9.1 Encoding

Encoding is the parsing model of section 2 run backwards: for each field, $raw = \text{round}((value - minimum) / resolution)$, then append that raw value as the stated number of bits, most significant bit first. For the rain threshold, first convert the interval you want in seconds into the transmitted form with $raw = 728 / \sqrt{t_seconds}$.

Two fields have no effect: the 2-bit type at the start and the 11-bit debug field at the end are both skipped by the gauge. Send zeros in both. They exist only to make the structure line up with MR2S.

9.2 Out-of-range values are replaced, not rejected

This is the opposite of what most command interfaces do, and the opposite of the MeteoHelix. If a field falls outside its range the firmware does NOT discard the command - it substitutes the FACTORY DEFAULT for that field and applies everything else. A snooze of 0 does not leave the old snooze in place; it silently resets the snooze to 600 seconds.

Field	Accepted range	Substituted if outside it
All-alarms snooze	120 - 61320 s	600 s
Rain click interval	0.506 - 529984 s	4.0 s

In practice every non-zero raw value encodes to something inside the accepted range, so the trap is the zero case: a snooze field of raw 0 decodes to 0 seconds, falls below the 120 second minimum, and resets the snooze. A rain interval of raw 0 is treated as a deliberate request for the default and resets the threshold to 4.0 seconds. Never send a partially filled command expecting the omitted field to be left alone - there is no such thing here.

9.3 Confirming a write

Because the command sends no reply, the only confirmation is to request the MR2S service message - downlink command 120 with payload 02 - and compare the settings it reports against what you sent. The two layouts share their first 21 bits, so the comparison is field for field.

9.4 Cloning settings between devices

A useful consequence of the shared layout: take an MR2S uplink, keep the first 4 of its 12 bytes, and you have a valid port 10 command that applies those exact settings to another device. The eight bytes dropped are the MR2S zero padding.

Step	Bytes
MR2S uplink received (12 B, port 2)	80AB60000000000000000000000000
First 4 bytes, sent as a port 10 downlink	80AB6000
Same command built from scratch	00AB6000

The two commands differ only in the leading 2 bits - the type field - which the firmware skips, so both apply identical settings. This was verified by running both through the firmware's own bit-extraction logic.

9.5 Worked examples

Settings	Downlink (4 B, port 10)	Base64
Snooze 600 s, alarm at a 4.0 s click interval (raw 364)	00AB6000	AKtgAA==
Snooze 1200 s, alarm at a 1.0 s click interval (raw 728)	0156C000	AVbAAA==

Field by field for the first: type 0; snooze $600/120 = 5$; interval $728/\sqrt{4.0} = 364$; debug 0. Packed as $00 + 000000101 + 0101101100 + 000000000000$. With a 0.2 mm gauge a 4.0 s click interval is a threshold of 180 mm/h and a 1.0 s interval is 720 mm/h.

10. Port 50 - command replies

Several configuration commands answer on port 50. The replies are bare hex fields with no message identifier, so in principle they can only be read against the command that was just sent. In this firmware, however, every reply has a distinct length, which is enough to tell them apart in practice.

Len	Sent in reply to	Layout	Fields
2 B	Port 112, time-sync counter, read or write	CCCC	Uplinks between time-sync requests
3 B	Port 111, rejoin settings, write	EE CCCC	Rejoin enabled 00/01; uplinks between rejoins
4 B	Port 121, rain counters, read or write	KKKK RRRR	Rain clicks 0-4095; intensity correction x 100
5 B	Port 111, rejoin settings, read	EE CCCC NNNN	Rejoin enabled; uplinks between rejoins; uplinks since the last one
6 B	Port 124, rain correction curve, read or write	LLLL HHHH cc dd	Lower and upper intensity breakpoints in CLICKS PER HOUR; corrections at each, in percent x 10

The two breakpoints in the 124 reply are rain intensities expressed in clicks per hour, not in mm/h and not in seconds. Multiply by the bucket size to read them as mm/h: the factory defaults of 350 and 3000 clicks/h are 70 and 600 mm/h on a 0.2 mm gauge.

10.1 Reading the rain counters back

Command 121 with payload 00 returns the current click counter and intensity correction without changing them. This is the reliable way to re-establish a baseline after a gap in the uplink series, or to confirm a counter that was set deliberately. The same 4-byte reply is returned when the command writes new values, so a write is self-confirming.

Example reply	Decodes to
001B0073	Rain clicks 27, intensity correction 1.15 clicks
015E0BB80043	Correction curve: 350 clicks/h at 0.0 %, 3000 clicks/h at 6.7 % - the factory defaults

11. Reference decoder

The decoder below is supplied alongside this document as the file `meteorain_gen2_decoder.js`. Use that file rather than retyping from this page.

It is written in ES5 with no dependencies, so it pastes directly into a The Things Stack v3 or ChirpStack v4 payload-formatter box; both call `decodeUplink({bytes, fPort})`. Set `RAIN_BUCKET_MM` at the top to the resolution of the rain gauge fitted before using it.

The decoder deliberately does NOT output a rainfall figure for a single payload, because no single payload contains one. Rainfall is the difference between two consecutive click counters, which is a job for the layer that holds the previous uplink, not for a stateless codec.

11.1 Test vectors

Feed these through any decoder before trusting it. They are generated from an encoder that mirrors the firmware's bit packer.

Port	Payload (hex)	Expected
1	0380DAD80E70	index 3, battery in the 3900-4100 mV slot, 27 clicks, shortest gap 4.00 s, correction 1.15, internal temp above 2 deg C, no debug flag
1	0700D80000002	index 7, battery not in slot, 27 clicks, no rain interval measured, correction 0.00, temp at or below 2 deg C, memory error flag set
1	0380DFFE0E70	as the first row but the rain interval is saturated at raw 1023 - 0.51 s, at or above 1421 mm/h on a 0.2 mm gauge
3	280DAD80	alarm index 5, 27 clicks, triggering interval 4.00 s (180 mm/h at 0.2 mm)
2	46420C002700BE440A000000	MR1S, MeteoRain, firmware 1.02.012, DevAddr 2700BE44, 10 minute period
2	80AB6000000000000000000000	MR2S, snooze 600 s, alarm threshold a 4.00 s click interval
2	049780C0D0D5975EC8028080	MR0S, the fixed not-implemented message of section 8 - discard it
4	0380DAD80E70	Same as the first row, flagged as a retransmission
50	001B0073	Port 121 reply: 27 clicks, correction 1.15
10 dwn	00AB6000	Alarm-settings downlink, section 9. Same settings as the MR2S row above. Pass to <code>decodeAlarmDownLink()</code> , not <code>decodeUplink()</code>

11.2 Source

```
// =====
// MeteoRain IoT Pro Gen2 - LoRaWAN uplink payload decoder
// Barani Design Technologies
//
// For firmware 1.02.012. See the Payload Decoding Guide for the field
// layouts and the reasoning behind them.
//
// Works as-is as a TTN v3 / ChirpStack v4 JavaScript codec: paste the whole
// file into the payload-formatter box. decodeUplink({bytes, fPort}) is the
// entry point both stacks call.
//
// Ports: 1 periodic (6 B), 2 service MR1S/MR2S (12 B), 3 alarm (4 B),
//        MR0S (type 0) also decodes, but it is NOT IMPLEMENTED - see below.
//        4 retransmission (6 or 4 B), 50 command reply (2-6 B).
//
// decodeAlarmDownlink(bytes) is also exported. It parses the 4-byte port 10
// alarm-settings DOWNLINK, for checking a command before sending it or
// reading one back from a log. TTN and ChirpStack never call it themselves.
// =====

// RAIN_BUCKET_MM: rain gauge resolution in mm per click - set to match the
// gauge fitted (0.1, 0.2 or 0.25). The device never transmits it.
var RAIN_BUCKET_MM = 0.2;
// ----- bit reader
function Reader(bytes) {
  this.b = bytes;
  this.pos = 0;
```

```

}
Reader.prototype.bits = function (n) {
  var v = 0;
  for (var i = 0; i < n; i++) {
    var byteNum = (this.pos + i) >> 3;
    var bitNum = 7 - ((this.pos + i) & 7);
    v = v * 2 + ((this.b[byteNum] >> bitNum) & 1);
  }
  this.pos += n;
  return v;
};

// Read a field and scale it. Returns null when every bit is set, which the
// firmware writes when a value is NaN or outside the encodable range.
function field(r, bits, min, res) {
  var raw = r.bits(bits);
  if (raw === Math.pow(2, bits) - 1) return null;
  return round4(raw * res + min);
}

// Same, but the all-ones pattern is a legitimate value (counters, flags, IDs).
function plain(r, bits, min, res) {
  var raw = r.bits(bits);
  if (min === undefined) return raw;
  return round4(raw * res + min);
}

function round4(x) {
  return Math.round(x * 10000) / 10000;
}

// 728/sqrt(t) -> seconds between rain clicks. 0 means "not measured".
function rainSeconds(raw) {
  if (raw === 0) return null;
  return round4((728 / raw) * (728 / raw));
}

// Rain rate in mm/h straight from the transmitted number:
// rate = 3600 * bucket * raw^2 / 728^2
function rainRate(raw, bucketMm) {
  if (raw === 0) return null;
  return round4((3600 * bucketMm * raw * raw) / 529984);
}

// ----- port 1 and 4
function decodePeriodic(r) {
  var o = {}, w = [];
  o.index = plain(r, 8);
  o.batteryInSlot = plain(r, 1);
  o.rainClicks = plain(r, 12);
  var rt = plain(r, 10);
  o.rainMinIntervalRaw = rt;
  o.rainMinInterval_s = rainSeconds(rt);
  o.rainPeakRate_mm_h = rainRate(rt, RAIN_BUCKET_MM);
  o.rainIntensityCorrection = field(r, 12, 0, 0.01);
  o.internalTempAbove2C = plain(r, 1);
  o.debugFlags = plain(r, 4);

  // NOTE: there is deliberately no "rain this period" field here. rainClicks
  // is a revolving lifetime counter; rainfall is the DIFFERENCE between two
  // consecutive uplinks. See the guide.

  // One payload cannot give a voltage - see the battery section of the guide.
  o.batteryHint_mV = o.batteryInSlot
    ? { min: 3300 + 200 * (o.index % 5), max: 3300 + 200 * (o.index % 5) + 200 }
    : null;

  // Decoded debug bits. The firmware assigns rather than ORs these, so only
  // the most recent one is ever present: the value is 0, 1 or 2, never 3.
  o.alarmSentInPeriod = (o.debugFlags & 1) === 1;
  o.eepromError = (o.debugFlags & 2) === 2;

  if (rt === 1023)
    w.push('rain interval saturated: clicks were faster than 0.51 s apart, ' +
      'the true rate is at or above this value');
  if (o.rainIntensityCorrection === null)
    w.push('rain intensity correction field is all-ones - value out of range');
  return { data: o, warnings: w };
}

// ----- port 3 and 4
function decodeAlarm(r) {
  var o = {}, w = [];
  o.index = plain(r, 5);
  o.rainClicks = plain(r, 12);
  var rt = plain(r, 10);
  o.rainIntervalRaw = rt;
  o.rainInterval_s = rainSeconds(rt);
  o.rainRate_mm_h = rainRate(rt, RAIN_BUCKET_MM);
  o.debugFlags = plain(r, 5);

```

```

    if (rt === 1023)
        w.push('rain interval saturated at the top of the 10-bit range');
    return { data: o, warnings: w };
}

// ----- port 2
var DEVICE_TYPE = ['test', 'MeteoHelix', 'MeteoWind', 'MeteoRain', 'MeteoAG'];

function decodeMr1s(r) {
    var o = { messageType: 'MR1S' };
    var hw = plain(r, 5);
    o.hardwareTypeValue = hw;
    o.hardwareType = DEVICE_TYPE[hw] !== undefined ? DEVICE_TYPE[hw] : 'unknown';
    var maj = plain(r, 3), min = plain(r, 6), pat = plain(r, 8);
    o.firmware = maj + '.' + pad2(min) + '.' + pad3(pat);
    o.debug = plain(r, 8); // always 0 in this firmware
    o.devAddr = hex8(plain(r, 32));
    o.payloadPeriod_min = plain(r, 8);
    // Remaining 24 bits are zero padding - do not read them.
    return { data: o, warnings: [] };
}

function decodeMr2s(r) {
    var o = { messageType: 'MR2S' };
    o.allAlarmsSnooze_s = plain(r, 9, 0, 120);
    var rt = plain(r, 10);
    o.rainClickIntervalRaw = rt;
    o.rainClickInterval_s = rainSeconds(rt);
    o.rainAlarmRate_mm_h = rainRate(rt, RAIN_BUCKET_MM);
    // Remaining 75 bits are zero padding - the 11-bit debug field named in
    // older documentation is not transmitted by this firmware.
    return { data: o, warnings: [] };
}

// MR0S - NOT IMPLEMENTED on this device. The sensors it was designed to report
// are not fitted; every field below is a compile-time constant and is
// identical on every unit. The firmware still answers a request for it
// (downlink 120 with parameter 00) with a well-formed frame, which is exactly
// what makes it dangerous - it looks like real telemetry. Decoded here only so
// a backend can recognise the message and throw it away.
function decodeMr0s(r) {
    var o = { messageType: 'MR0S', implemented: false };
    // Bits 3-66 are fixed constants that were never measured, so they are SKIPPED
    // rather than decoded - there is nothing here worth handing to a caller.
    r.bits(32); r.bits(32);
    o.inclinationX_deg = plain(r, 8, 0, 0.25);
    o.inclinationY_deg = plain(r, 8, 0, 0.25);
    o.inclinationZ_deg = plain(r, 8, 0, 0.25);
    o.debugFlags = plain(r, 6);
    return {
        data: o,
        warnings: ['MR0S is NOT IMPLEMENTED on the MeteoRain IoT Pro Gen2. Bits 3-66 ' +
            'are not decoded at all, and the inclination values above are ' +
            'fixed constants, not measurements. Discard this message.'];
    };
}

// ----- port 10 downlink
// The alarm-settings DOWNLINK (4 bytes). Not an uplink - this is here so a
// backend can check what it is about to send, or read one back from a log.
// Its 32 bits are the same fields as the first 4 bytes of MR2S, followed by
// an 11-bit debug field; the firmware ignores both the type and debug fields.
//
// Out-of-range values are NOT rejected here: the firmware silently replaces
// them with factory defaults and applies the rest. See rangeCheck().
function decodeAlarmDownlink(bytes) {
    if (bytes.length !== 4) {
        return { errors: ['alarm downlink needs 4 bytes, got ' + bytes.length] };
    }
    var r = new Reader(bytes), o = {};
    o.typeField = plain(r, 2); // ignored by the firmware
    o.allAlarmsSnooze_s = plain(r, 9, 0, 120);
    var rt = plain(r, 10);
    o.rainClickIntervalRaw = rt;
    o.rainClickInterval_s = rt === 0 ? 4.0 : rainSeconds(rt); // 0 = default
    o.rainAlarmRate_mm_h = rainRate(rt, RAIN_BUCKET_MM);
    o.debugField = plain(r, 11); // ignored by the firmware
    return { data: o, warnings: rangeCheck(o) };
}

// The firmware does not reject a bad field, it substitutes a default and
// keeps going - so a wrong value is applied, not ignored. Check before sending.
function rangeCheck(o) {
    var w = [];
    if (o.allAlarmsSnooze_s < 120 || o.allAlarmsSnooze_s > 61320)
        w.push('snooze ' + o.allAlarmsSnooze_s + ' s outside 120-61320: the ' +
            'device would REPLACE it with the 600 s default');
}

```

```

    if (o.rainClickIntervalRaw === 0)
        w.push('rain interval 0 selects the 4.0 s default');
    return w;
}

// ----- port 50
// Replies to configuration commands. They carry no command identifier, but in
// this firmware every reply has a distinct length, so they can be told apart.
function decodeCommandReply(b) {
    var h = bytesToHex(b), o = { raw: h };
    if (b.length === 2) { // port 112
        o.reply = 'timeRequestCounter';
        o.timeReqCounter = (b[0] << 8) | b[1];
    } else if (b.length === 3) { // port 111 set
        o.reply = 'rejoinSettings';
        o.rejoinEnabled = b[0] === 1;
        o.rejoinCounter = (b[1] << 8) | b[2];
    } else if (b.length === 4) { // port 121
        o.reply = 'rainCountersReadback';
        o.rainClicks = (b[0] << 8) | b[1];
        o.rainIntensityCorrection = round4(((b[2] << 8) | b[3]) / 100);
    } else if (b.length === 5) { // port 111 get
        o.reply = 'rejoinStatus';
        o.rejoinEnabled = b[0] === 1;
        o.rejoinCounter = (b[1] << 8) | b[2];
        o.uplinksSinceRejoin = (b[3] << 8) | b[4];
    } else if (b.length === 6) { // port 124
        // The two breakpoints are rain intensities in CLICKS PER HOUR, not mm/h
        // and not seconds - convert with the gauge resolution to read them in mm/h.
        o.reply = 'rainCorrectionCurve';
        o.intensityLow_clicks_h = (b[0] << 8) | b[1];
        o.intensityHigh_clicks_h = (b[2] << 8) | b[3];
        o.intensityLow_mm_h = round4(o.intensityLow_clicks_h * RAIN_BUCKET_MM);
        o.intensityHigh_mm_h = round4(o.intensityHigh_clicks_h * RAIN_BUCKET_MM);
        o.correctionLow_pct = round4(b[4] / 10);
        o.correctionHigh_pct = round4(b[5] / 10);
    } else {
        o.reply = 'unknown';
    }
    return {
        data: o,
        warnings: ['port 50 reply identified by length - confirm it against the ' +
            'command you sent']
    };
}

function pad2(n) { return (n < 10 ? '0' : '') + n; }
function pad3(n) { return (n < 10 ? '00' : n < 100 ? '0' : '') + n; }
function hex8(n) {
    var s = (n >>> 0).toString(16).toUpperCase();
    while (s.length < 8) s = '0' + s;
    return s;
}

// ----- dispatch
function decodeUplink(input) {
    var b = input.bytes, port = input.fPort;
    var r = new Reader(b);
    var res;
    if (port === 1) {
        if (b.length !== 6) return err('port 1 needs 6 bytes, got ' + b.length);
        res = decodePeriodic(r);
    } else if (port === 3) {
        if (b.length !== 4) return err('port 3 needs 4 bytes, got ' + b.length);
        res = decodeAlarm(r);
    } else if (port === 4) {
        // A replay of a port 1 or port 3 payload. Length is the only discriminator.
        if (b.length === 6) res = decodePeriodic(r);
        else if (b.length === 4) res = decodeAlarm(r);
        else return err('port 4 needs 6 or 4 bytes, got ' + b.length);
        res.data.retransmitted = true;
    } else if (port === 2) {
        // MR0S, MR1S and MR2S are all 12 bytes - the leading 2 bits tell them apart.
        if (b.length !== 12) return err('port 2 needs 12 bytes, got ' + b.length);
        var type = r.bits(2);
        if (type === 0) res = decodeMr0s(r); // NOT IMPLEMENTED - see above
        else if (type === 1) res = decodeMr1s(r);
        else if (type === 2) res = decodeMr2s(r);
        else return err('unknown service message type ' + type);
    } else if (port === 50) {
        res = decodeCommandReply(b);
    } else {
        return err('unhandled port ' + port);
    }
    return { data: res.data, warnings: res.warnings };
}

function err(m) { return { data: {}, errors: [m] }; }

```

```
function bytesToHex(b) {
  var s = '';
  for (var i = 0; i < b.length; i++) {
    var h = b[i].toString(16).toUpperCase();
    s += (h.length < 2 ? '0' : '') + h;
  }
  return s;
}

// Node/CommonJS export - harmless in TTN and ChirpStack, which ignore it.
if (typeof module !== 'undefined') { module.exports = { decodeUplink: decodeUplink, decodeAlarmDownlink: decodeAlarmDownlink }; }
```



12. Migrating an existing decoder

If you already decode an earlier MeteoRain, or built a decoder from an older payload sheet, these are the points where that decoder will be wrong against firmware 1.02.012. Field positions and widths in the periodic and alarm payloads are unchanged.

12.1 The battery bit is probably being decoded as a voltage

Older payload sheets describe the battery bit as a voltage selected by index modulo 10, in 0.1 V steps, and note that "every other message will have the correct voltage". That is not what this firmware does. It tests whether the voltage lies inside one 200 mV slot chosen by $\text{index} \% 5$, and reports a single yes or no - see 3.3.

A decoder written from the older description publishes a plausible voltage from every uplink, and it is wrong on four uplinks out of five. Collect five consecutive indices and report the slot that answered 1, or report nothing.

12.2 The alarm index revolves at 32, not 64

Older documentation gives the alarm message counter as revolving 0 to 63. The field is 5 bits and the firmware stores 32 alarm payloads, so it revolves 0 to 31. A retransmission request for an index of 32 or above is rejected and produces no uplink at all.

12.3 The intensity correction never exceeds 10.23

The field is 12 bits at 0.01, so an older sheet correctly gives its span as 0 to 40.95. The firmware, however, wraps the accumulator at 10.23, so only the bottom quarter of the range is ever used. Wrap-detection logic that waits for 40.95 will never fire and will report a large negative correction each time the accumulator revolves.

12.4 Smaller points

Item	Note
MR2S debug flags	Older documentation shows an 11-bit debug field at bits 22-32. The firmware does not write it, so it always reads zero. Harmless to decode, but do not length-check or branch on it
Rain time formula	Some older notes describe the rain field as $728/\text{time}^2$. It is $728/\sqrt{\text{time}}$, so the interval is $(728/\text{raw})^2$ - see 3.2. The two give very different rain rates
MR1S version field widths	Older notes give the minor revision as 0-255 and the patch as 0-256. The fields are 6 and 8 bits, so the real limits are 0-63 and 0-255
MR1S sensor serial	The MeteoHelix MR1S/MH1S layout carries sensor and solar serial numbers. The MeteoRain layout does not - bits 33-64 are the DevAddr, followed by the payload period. Do not reuse a MeteoHelix service decoder here
MR0S	NOT IMPLEMENTED - the sensors this message was designed to report are not fitted to this device. The firmware still answers a request for it with fixed constants. See section 8 before publishing anything from it
Alarm payload debug flags	Five bits (28-32), always zero in this firmware

13. Decoder checklist

#	Check
1	Read bit ranges, not bytes
2	Treat rain clicks as a lifetime counter and difference it, handling the 4095 wrap - never publish a single payload's counter as rainfall
3	Difference the intensity correction too, wrapping at 10.23, and add it to the click difference

#	Check
4	Convert rain time with $(728/\text{raw})^2$ and treat raw 0 as not measured
5	Report raw 1023 on a rain time field as a lower bound, not an exact rate
6	Set the rain bucket size to match the gauge; it is not in the payload
7	Collect five consecutive indices before reporting a battery voltage
8	Read bit 44 as a frost warning, not a temperature - and not as a sensor-health flag
9	Expect periodic debug values of 0, 1 or 2 only - the firmware does not combine them
10	Branch port 2 on the leading 2 bits, not on length - MR0S, MR1S and MR2S are all 12 bytes
11	Discard MR0S - it is not implemented and carries fixed constants, not measurements
12	Decode port 4 by length: 6 bytes periodic, 4 bytes alarm
13	Expect the alarm index to revolve at 32, not 64
14	When ENCODING the port 10 downlink, range-check every field first - an out-of-range field is silently REPLACED with a factory default, and a zero snooze resets the snooze rather than leaving it alone